Mediator is a behavioral design pattern that lets you reduce chaotic dependencies between objects. The pattern restricts direct communications between the objects and forces them to collaborate only via a mediator object.

Pilots of different airplanes don't talk directly to each other to avoid collisions. Instead, every pilot talks to the Air Traffic Control Tower. The tower (Mediator) coordinates all flights, ensuring safety and order

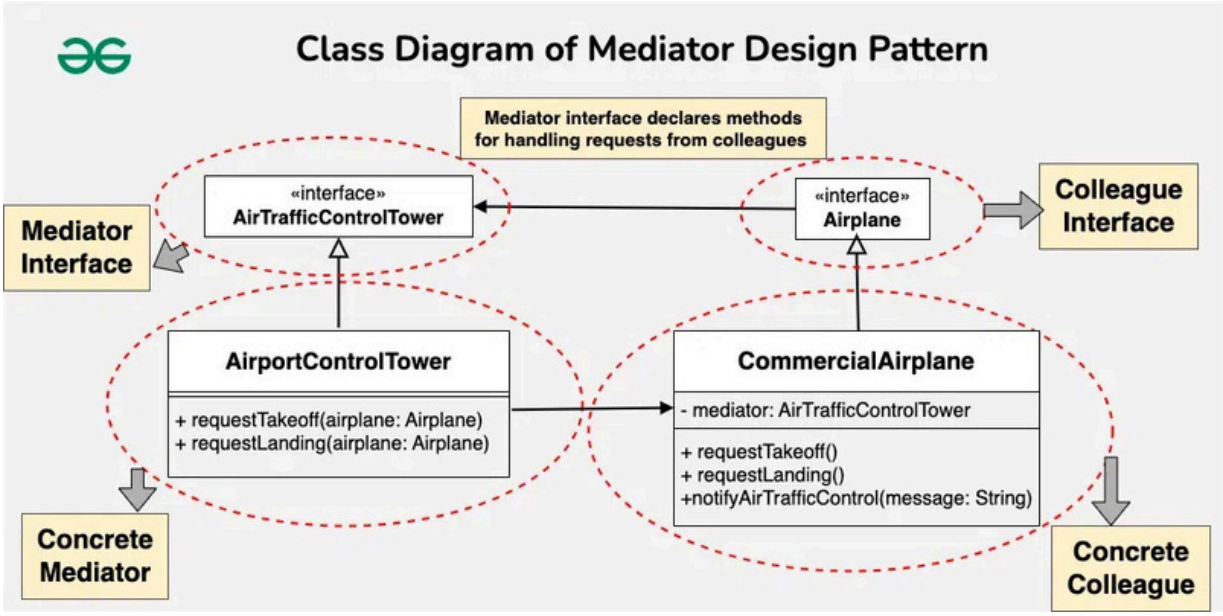
```java
import java.util.ArrayList;
import java.util.List;

// Mediator Interface
interface ChatMediator {
    void sendMessage(String msg, User user);
    void addUser(User user);
}

// Colleague Class
abstract class User {
    protected ChatMediator mediator;
    protected String name;

    public User(ChatMediator med, String name) {
        this.mediator = med;
        this.name = name;
    }

    public abstract void send(String msg);
    public abstract void receive(String msg);
}

// Concrete Mediator
class ChatRoom implements ChatMediator {
    private List<User> users = new ArrayList<>();
```

```java
    @Override
    public void addUser(User user) {
        this.users.add(user);
    }

    @Override
    public void sendMessage(String msg, User user) {
        for (User u : users) {
            // Message should not be received by the sender
            if (u != user) {
                u.receive(msg);
            }
        }
    }
}

// Concrete Colleague
class ChatUser extends User {
    public ChatUser(ChatMediator med, String name) {
        super(med, name);
    }

    @Override
    public void send(String msg) {
        System.out.println(this.name + " sends: " + msg);
        mediator.sendMessage(msg, this);
```

```java
    }

    @Override
    public void send(String msg) {
        System.out.println(this.name + " sends: " + msg);
        mediator.sendMessage(msg, this);
    }

    @Override
    public void receive(String msg) {
        System.out.println(this.name + " received: " + msg);
    }
}

// Test Class
public class MediatorPatternTest {
    public static void main(String[] args) {
        ChatMediator chatRoom = new ChatRoom();

        User user1 = new ChatUser(chatRoom, "Alice");
        User user2 = new ChatUser(chatRoom, "Bob");
        User user3 = new ChatUser(chatRoom, "Charlie");

        chatRoom.addUser(user1);
        chatRoom.addUser(user2);
        chatRoom.addUser(user3);
```

Why this is a Mediator Pattern

- **Decoupling:** `User` objects no longer need to know about other `User` objects. They only know the `ChatMediator`.
- **Centralized Control:** The logic for how messages are distributed is contained entirely within the `ChatRoom` class.
- **Simplified Maintenance:** If you want to change how users interact (e.g., adding a profanity filter or private messaging), you only modify the `ChatRoom` class.
- **Many-to-Many Reduction:** It transforms complex many-to-many relationships between objects into simpler one-to-many relationships between the mediator and its colleagues.



1. Primary Intent

- **Mediator:** Focuses on **centralizing complex communication** between various objects (colleagues). It prevents objects from talking directly to each other, forcing them to go through a "hub."
- **Observer:** Focuses on **distributed notification**. It establishes a one-to-many dependency where one object (the subject) changes state, and all its dependents (observers) are notified automatically.

2. Communication Flow

- **Mediator (Many-to-Many):** It facilitates bidirectional communication. Object A talks to the Mediator, which then talks to Object B, C, or D. It manages the *workflow*.
- **Observer (One-to-Many):** It is usually unidirectional. The Subject doesn't care who is listening; it just shouts "I've changed!" and the Observers react.